# Architecture de la Base de Données

## Microservice Document Management — MyKyntus

# 1. Introduction

**Dans le cadre de l'architecture microservices de l'application MyKyntus, le microservice Document Management est responsable de la gestion des documents administratifs des collaborateurs.**

**Ce microservice permet notamment :**

- **la gestion des demandes de documents**
- **la génération et le stockage des documents**
- **la gestion des templates de documents**
- **la traçabilité des actions effectuées**
- **la gestion des notifications**
- **la gestion des paramètres utilisateur liés aux documents**

**L'architecture de la base de données a été conçue afin d'assurer :**

- **la scalabilité**
- **la traçabilité**
- **la sécurité**
- **la maintenabilité du système**

# 2. Approche Architecture Microservices

**Dans une architecture microservices, chaque service possède sa propre base de données.**

**Ainsi, le microservice Document Management possède sa base de données dédiée :**

**document_management_db**

**Les autres microservices (User Service, Notification Service, etc.) interagissent avec ce service via API et événements, et non par accès direct à la base de données.**

**Cette approche permet :**

- **l'indépendance des services**
- **la réduction du couplage entre les composants**
- **une meilleure évolutivité du système.**

# 3. Modèle Conceptuel de Données (ERD)

**Le modèle de données repose sur plusieurs entités principales qui représentent les objets métier du système.**

**Les principales entités identifiées sont :**

- **Documents**
- **DocumentRequests**
- **DocumentTypes**
- **DocumentTemplates**
- **DocumentLogs**
- **Notifications**
- **UserSettings**

**Ces entités sont reliées entre elles selon des relations définies dans le diagramme ERD (Entity Relationship Diagram).**

# 4. Description des Entités

## 4.1 Table Documents

**La table Documents stocke les documents générés et disponibles pour les utilisateurs.**

**Elle contient les informations principales relatives aux documents.**

**Principaux attributs :**

- **DocumentId : identifiant unique du document**
- **Title : nom du document**
- **DocumentTypeId : type du document**
- **TemplateId : template utilisé pour générer le document**
- **EmployeeId : utilisateur propriétaire du document**
- **FileUrl : chemin de stockage du document**
- **Version : version du document**
- **Status : statut du document**
- **GeneratedBy : utilisateur ayant généré le document**
- **CreatedAt : date de création**
- **UpdatedAt : date de modification**
- **DeletedAt : suppression logique (soft delete)**

## 4.2 Table DocumentRequests

**La table DocumentRequests gère les demandes de documents effectuées par les utilisateurs.**

**Principaux attributs :**

- **RequestId : identifiant unique de la demande**

- **EmployeeId : utilisateur ayant effectué la demande**
- **DocumentTypeId : type de document demandé**
- **Status : statut de la demande**
- **Comment : commentaire associé**
- **RequestedAt : date de la demande**
- **ApprovedBy : utilisateur ayant validé la demande**
- **ApprovedAt : date de validation**
- **CreatedAt : date de création**
- **UpdatedAt : date de modification**
- **DeletedAt : suppression logique**

## 4.3 Table DocumentTypes

**La table DocumentTypes définit les différents types de documents disponibles dans le système.**

**Exemples :**

- **Attestation de travail**
- **Certificat de salaire**
- **Certificat de formation**
- **Contrat de travail**

**Principaux attributs :**

- **DocumentTypeId : identifiant du type**
- **Name : nom du type**
- **Description : description**
- **RequiresApproval : indique si une validation RH est requise**
- **CreatedAt : date de création**
- **UpdatedAt : date de modification**

## 4.4 Table DocumentTemplates

**La table DocumentTemplates stocke les modèles utilisés pour générer les documents.**

**Principaux attributs :**

- **TemplateId : identifiant du template**
- **Name : nom du template**
- **DocumentTypeId : type de document associé**
- **TemplateFile : fichier modèle**
- **Variables : variables dynamiques du template**
- **Version : version du template**
- **CreatedAt : date de création**
- **UpdatedAt : date de modification**

**Les templates utilisent des variables dynamiques telles que :**

- EmployeeName
- Department
- Position
- Date

## 4.5 Table DocumentLogs

La table DocumentLogs assure la traçabilité des actions effectuées sur les documents.

Elle permet d'enregistrer les opérations telles que :

- génération de document
- téléchargement de document
- modification
- suppression

Principaux attributs :

- LogId : identifiant du log
- DocumentId : document concerné
- UserId : utilisateur ayant effectué l'action
- Action : type d'action
- CreatedAt : date de l'action

## 4.6 Table Notifications

La table Notifications stocke les notifications envoyées aux utilisateurs.

Ces notifications peuvent être générées lors de différents événements :

- approbation d'une demande
- rejet d'une demande
- génération d'un document
- notification système

Principaux attributs :

- NotificationId : identifiant
- UserId : utilisateur destinataire
- Title : titre de la notification
- Message : contenu
- Type : type de notification
- IsRead : statut de lecture
- CreatedAt : date de création

## 4.7 Table UserSettings

La table UserSettings contient les préférences utilisateur liées au module documents.

**Principaux attributs :**

- **SettingId : identifiant**
- **UserId : utilisateur concerné**
- **Language : langue de l'interface**
- **Theme : thème d'affichage (Dark ou Light)**
- **EmailNotifications : notifications email activées**
- **SystemNotifications : notifications système activées**
- **CreatedAt : date de création**
- **UpdatedAt : date de modification**

# 5. Relations entre les Entités

**Les relations principales entre les tables sont les suivantes :**

- **Un utilisateur peut effectuer plusieurs DocumentRequests**
- **Un utilisateur peut posséder plusieurs Documents**
- **Un utilisateur peut recevoir plusieurs Notifications**
- **Un utilisateur possède un seul UserSettings**
- **Un DocumentType peut être associé à plusieurs documents**
- **Un DocumentTemplate peut générer plusieurs documents**
- **Une DocumentRequest peut produire un document**
- **Un Document peut avoir plusieurs logs**

**Ces relations sont représentées dans le diagramme ERD.**

# 6. Bonnes Pratiques de Conception

**La conception de la base de données respecte plusieurs bonnes pratiques :**

## Utilisation des UUID

**Les identifiants utilisent le type UUID afin de garantir l'unicité des enregistrements dans un environnement distribué.**

## Audit Fields

**Les champs suivants sont utilisés pour assurer la traçabilité :**

- **CreatedAt**
- **UpdatedAt**

## Soft Delete

**Le champ DeletedAt permet d'effectuer une suppression logique des données sans les supprimer définitivement.**

### Versioning

**Les documents et templates utilisent un système de version afin de conserver l'historique des modifications.**

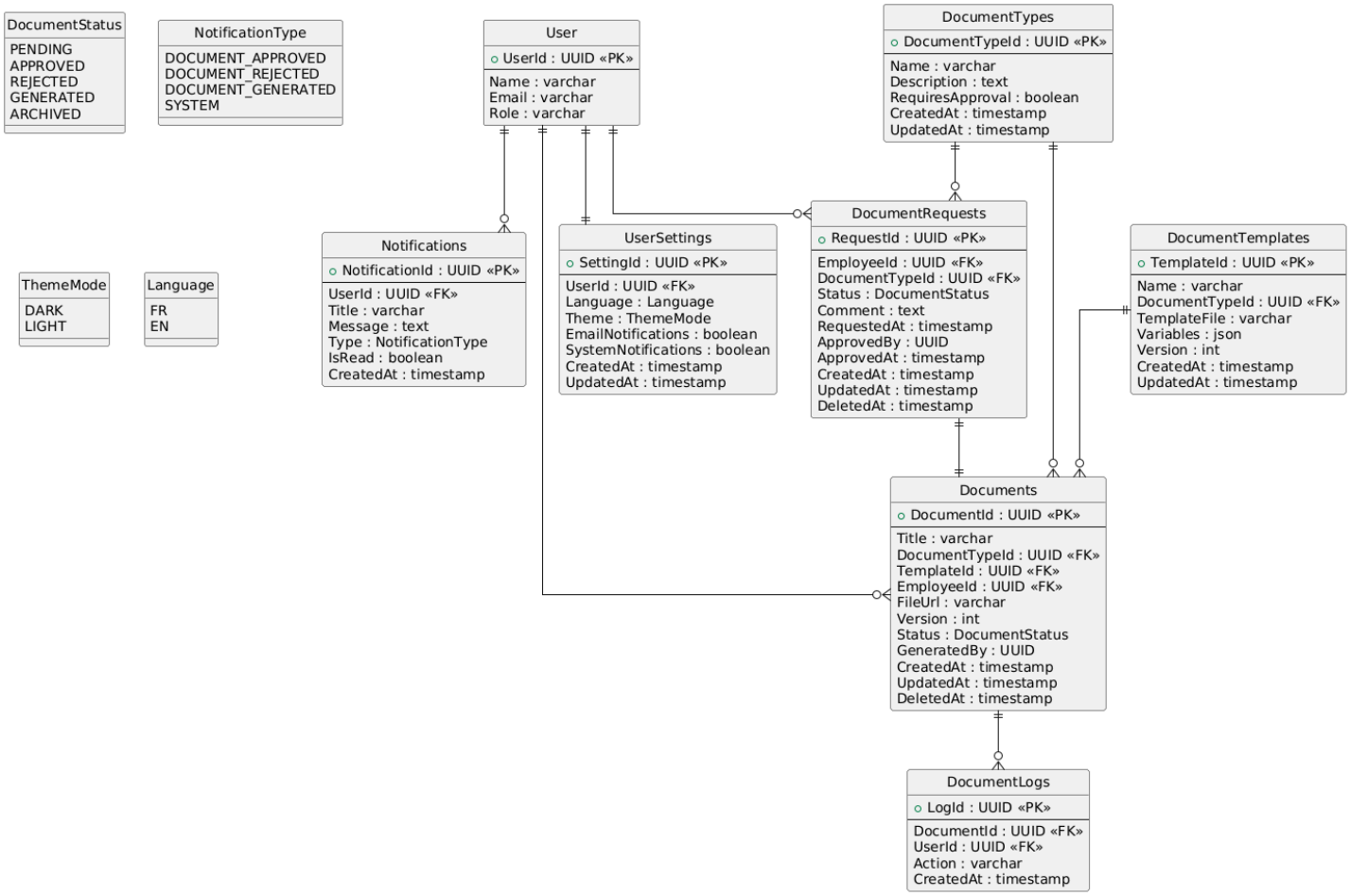
# 7. Conclusion

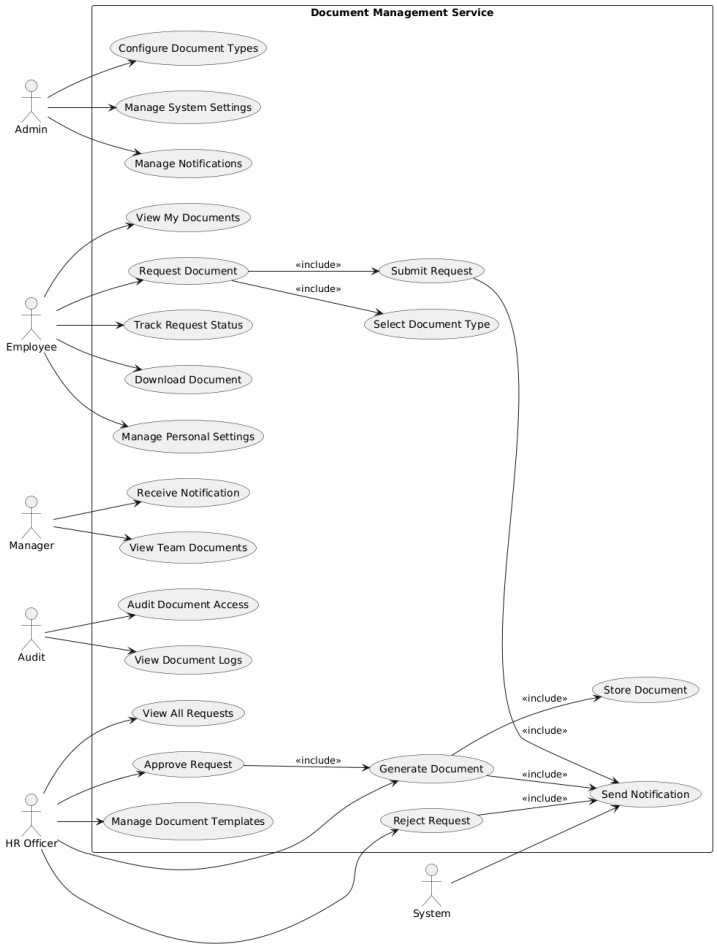
**L'architecture de la base de données du microservice Document Management a été conçue afin de répondre aux exigences fonctionnelles et techniques de l'application MyKyntus.**

**Elle permet :**

- **une gestion structurée des documents**
- **une traçabilité complète des actions**
- **une gestion flexible des templates**
- **une gestion efficace des notifications**
- **une personnalisation des paramètres utilisateur**

**Cette architecture constitue une base solide pour le développement et l'évolution future du système.**




Document Management Service
Configure Document Types
Manage System Settings
Manage Notifications
Admin
View My Documents
Request Document
«include»
Submit Request
«include»
Select Document Type
Track Request Status
Download Document
Manage Personal Settings
Employee
Receive Notification
View Team Documents
Manager
Audit Document Access
View Document Logs
Audit
View All Requests
Approve Request
«include»
Generate Document
«include»
Store Document
«include»
Send Notification
«include»
Reject Request
«include»
Manage Document Templates
HR Officer
System